TP4 : automatisation et scripting

Machines et commandes

Deux VM dédiées, `tp4-web1` et `tp4-web2`, ont été utilisées le 28 octobre 2026. Elles sont distinctes du cluster du TP3.

```
python3 lab/manage.py start tp4-web1
python3 lab/manage.py start tp4-web2
python3 lab/manage.py wait tp4-web1
python3 lab/manage.py wait tp4-web2
tar -cf - tp4 | python3 lab/manage.py ssh tp4-web1 'tar -xf -'
python3 lab/manage.py ssh tp4-web1 'sudo bash tp4/install-web.sh'
.venv/bin/ansible-playbook -i tp4/inventory.ini tp4/playbook.yml
.venv/bin/ansible-playbook -i tp4/inventory.ini tp4/playbook.yml
python3 tp4/verify.py
```

Le script Bash a d'abord été exécuté sur le premier serveur. Le playbook a ensuite été exécuté sur les deux serveurs dans un même play, avec cinq connexions parallèles disponibles (`forks = 5`). Les lignes intercalées des deux hôtes dans les journaux montrent leur prise en charge pendant la même exécution.

Script Bash

install-web.sh vérifie les droits root, installe Nginx, copie index.html, crée le site, retire le site par défaut, valide la configuration, active et démarre le service. Les erreurs interrompent l'exécution avec `set -euo pipefail`.

Les fichiers modifiés sont `/var/www/ad10-static/index.html`, `/etc/nginx/sites-available/ad10-static` et le lien dans `sites-enabled`.

Playbook et rôle de pare-feu

playbook.yml exprime le même état avec les modules `apt`, `file`, `copy`, `template` et `service`. Le rôle static_web gère Nginx. Les modifications de configuration déclenchent une validation, puis un rechargement, par handlers.

Le rôle réutilisable firewall installe nftables, valide les règles avant leur installation et active le service au démarrage. Les entrées sont bloquées par défaut. SSH et HTTP sont autorisés ; le bouclage, les réponses aux connexions et le DHCP de la carte de sortie sont préservés. Le rôle accepte aussi des règles limitées à des adresses sources pour le projet final.

Comparaison

Critère	Bash	Ansible
Description	Suite de commandes à exécuter.	État attendu des ressources.

Critère	Bash	Ansible
Exécution distante	Nécessite une connexion et une copie explicites.	Inventaire SSH et exécution sur plusieurs hôtes.
Réexécution	Réinstalle ou réécrit selon la logique du script.	Ne modifie que les ressources différentes.
Redémarrage	Appels explicites à systemctl.	Handlers déclenchés par les modifications.
Erreurs	Arrêt du script et sortie de commande.	Résultat détaillé par tâche et par hôte.
Organisation	Adapté à une installation courte.	Rôles, variables et inventaires réutilisables.

L'idempotence dépend du contenu des tâches ; Ansible ne la garantit pas pour une commande arbitraire. Ici, les modules déclaratifs permettent une seconde exécution avec `changed=0` sur les deux machines.

Vérifications et résultats

Les deux pages ont répondu en HTTP 200 sur localhost:8041 et localhost:8042. L'en-tête `X-Backend` distingue les deux machines. Nginx et nftables sont actifs et activés au démarrage. Le contrôle de la seconde exécution confirme `changed=0`, `unreachable=0` et `failed=0` pour chaque hôte.

- Exécution réelle du script Bash.
- Première exécution Ansible.
- Seconde exécution Ansible.
- Résultats du contrôle.

Aucune erreur d'exécution n'a été observée sur ces deux VM pendant le déploiement du TP4.